

Sanjana Singh & Skyla Fitzgerald

Spring 2026 NLP Homework 2: N-grams and Naive Bayes

Naive Bayes Theory

Question 1: NB Classification and Bayes' Rule

(a) Start with the objective: $\operatorname{argmax}_{c \in C} P(c|d)$

↳ 1. Apply Bayes' rule: $P(c|d) = \frac{P(d|c) \cdot P(c)}{P(d)}$ ← Bayes' Theorem

↳ 2. Substitute into the argmax : $\operatorname{argmax}_{c \in C} \frac{P(d|c)P(c)}{P(d)}$

- We substitute using Bayes' Rule, which expresses the posterior probability $P(c|d)$ in terms of likelihood $P(d|c)$, prior $P(c)$, and evidence $P(d)$.

↳ 3. Remove denominator: $\operatorname{argmax}_{c \in C} P(d|c) \cdot P(c)$

- We remove the denominator here because the term $P(d)$ [$P(d)$ is the probability of the document occurring, which is independent of the class c . So, $\operatorname{argmax}_{c \in C} \frac{P(d|c)P(c)}{P(d)} = \operatorname{argmax}_{c \in C} P(d|c)P(c)$] is the same for all classes. Since $P(d) > 0$ and is constant for all $c \in C$, it acts as a scaling factor that does not change which value is largest (max).

(b) We typically move to log-space in practice for computational stability (avoiding numerical underflow). Naive Bayes multiplies many small probabilities when computing the term $P(d|c)$. Multiplying many small numbers can produce values extremely close to 0, which computers cannot represent accurately (underflow), leading them to round to 0 and lose precision. Taking the log converts multiplication into addition, which is more numerically stable, faster, and more efficient for computers to compute. In addition, logarithm is a strictly increasing function, meaning the class with highest probability will also have the highest log-probability.

Question 2: Independence Assumptions in Sentiment Classification

- (a) The phrase "almost enjoyable" creates problems for a Naive Bayes classifier because Naive Bayes relies on both conditional Independence and bag-of-words assumption. Conditional Independence refers to the fact that the model assumes words are independent of each other. This means it treats the word "almost" and "enjoyable" as independent pieces of evidence. However, in the English language, the word "almost" changes the meaning of "enjoyable," where "enjoyable" is strongly positive while "almost enjoyable" is not as strong and possibly a neutral sentiment. Bag-of-words assumption means the model ignores word order and treats words as separate features. The model sees the two words "almost" and "enjoyable" but does not understand the "almost" directly modifies "enjoyable". The model ignores the order entirely and only sees the individual words, so it cannot represent that "almost" affects "enjoyable". As a result the strong positive probability of "enjoyable" may dominate, causing the classifier to incorrectly predict a positive sentiment, even though the phrase may not be strongly positive.
- (b) To handle modifiers like "almost", we can use n-grams, specifically bigrams, to include word pairs as features rather than only single words. The classifier would include "almost enjoyable" as a feature and learn its probability given each sentiment class $P(\text{almost enjoyable} | c)$. By treating "almost enjoyable" as a single bigram feature, the model can learn a distinct conditional probability $P(\text{almost enjoyable} | \text{positive})$ that is significantly lower than $P(\text{enjoyable} | \text{positive})$. This allows the model to capture local context and dependencies that are lost with the unigram Bag-of-words assumption.

Question 3: From Chain Rule to Bigram Approximation

(a) Using the first-order Markov assumption:

$$P(w_1, w_2, \dots, w_n) \approx P(w_1) \cdot P(w_2 | w_1) \cdot P(w_3 | w_2) \cdot \dots \cdot P(w_n | w_{n-1})$$
$$\hookrightarrow \approx P(w_1) \prod_{[from\ k=2\ to\ n]} P(w_k | w_{k-1})$$

(b) The information that is discarded when moving from the full chain rule to the bigram model is the conditional history beyond a window of size 1. The full chain rule uses the entire history $P(w_k | w_1, \dots, w_{k-1})$, meaning, in order to predict the next word, the model uses all previous words in the sentence as context. The bigram reduces this to $P(w_k | w_{k-1})$, which discards information about all words except the immediately previous one, since the first-order Markov assumption assumes only the immediately previous word matters. In linguistic terms, this means the model cannot capture long-distance dependencies or context that appears earlier in the sentence. For example,

"The soup I cooked today was good." The correct verb "was" depends on "soup", not just the previous word "today." The bigram model only sees $P(\text{was} | \text{today})$ and ignores the earlier subject.

(c) Bigram models work well when sentences or phrases are short and the most useful information is local. Some examples include "thank you" and "good morning" since these phrases are highly predictable local dependencies, where the second word is almost entirely determined by the first, making the conditional history irrelevant. Bigrams also work well with domain-specific or repetitive language, like in phrases such as "partly cloudy skies" or "chance of rain," since these domains use local grammar and predictable word combinations.

Question 4: Experimental Design

- (a) Including test sentences in the training corpus artificially lowers perplexity because it rewards memorization instead of generalization. Perplexity depends directly on the sum of log probabilities of test words. For sentences used in training that are also used in testing, the model has already seen those exact same word sequences and would assign artificially high probabilities. As a result, the average log probability increases and the computed perplexity decreases, giving the appearance of a strong performance. However, this low perplexity does not reflect how well the model handles new, unseen sentences.
- (b) The practice of re-evaluation on the test set is invalid because the test set must include unseen data. If the researcher uses the test set to tune parameters, then the test set is no longer an independent measure of how the model generalizes. Instead, the test set becomes part of the training process, which leads to overfitting of the test set. Because of this, the reported perplexity no longer shows the true performance, but rather a biased one.
- (c) The development set in n-gram language modeling is used for tuning parameters like smoothing parameters, choosing model order, using backoff or interpolation, comparing model variants, etc. It is used after training but before testing. This allows researchers to improve the model while keeping the test set unbiased and unseen.

Question 5: Model Order, Perplexity, and Generalization

- (a) Perplexity decreases as the n -gram order increases from unigram to trigram because of the nature of unigrams and trigrams. A unigram model assumes $P(w_k)$, which ignores context completely, so it cannot use surrounding words to make better predictions. Because perplexity measures how well a model predicts a sequence, unigrams lead to high uncertainty and therefore high perplexity. A bigram model uses one word of context $P(w_k | w_{k-1})$, which reduces uncertainty because the previous word may provide useful information the model could use to predict the next word. As a result, the perplexity decreases compared to the unigram. The perplexity decreases even more when moving to trigrams because a trigram model uses two words of context $P(w_k | w_{k-2}, w_{k-1})$. With more context, predictions become more accurate, uncertainty decreases further, and perplexity drops even more. Essentially, increasing n -gram order may improve predictions because more contextual information is used, which lowers average uncertainty.
- (b) A 5-gram model might perform worse than a trigram model because, although it uses more context, it also requires estimating probabilities for many more possible word sequences. This means that, especially on a smaller corpus, many 5-grams will rarely or never appear. This leads to unreliable probability estimates, even with smoothing, because long contexts are rarely observed. There is also the problem of overfitting, because a 5-gram model may memorize specific training sequences rather than learning general patterns, which would perform poorly on unseen test data.